

Programming Language Principles, Design, and Implementation

Compilers - Functional Programming Languages

Vincent Rahli

University of Birmingham

Where are we?

- ▶ Part 1: Principles of programming
- ▶ **Part 2: Compilers**

Today

Functional Programming Languages

- ▶ Closure conversion
- ▶ CPS transformation
- ▶ Defunctionalization
- ▶ Krivine's abstract machine

Further reading:

- ▶ “A Functional Correspondence between Evaluators and Abstract Machines” by Mads Sig Ager, Dariusz Biernacki, Olivier Danvy, and Jan Midtgaard

Problem

TAC can be seen as an abstract machine for **imperative programming** languages (e.g., C).

An **abstract machine** is

- ▶ an intermediate languages
- ▶ with a small-step operational semantics

Functional programming languages (e.g., Haskell), have an additional difficulty: **higher-order functions**.

Numerous abstract machines have been defined for the λ -calculus, such as the SECD, CAM, CEK, Krivine machines.

In this lecture we will see how to transform a call-by-name evaluator for the λ -calculus to a Krivine machine using “Danvy’s methodology”.

Source Language

Recall the syntax of the λ -calculus:

$$M \in \text{LC} ::= x \mid \lambda x.M \mid M M$$

Krivine's machine makes use of **de Bruijn indices**. We use names instead for simplicity.

A call-by-name interpreter for the λ -calculus can be defined as follows:

$$\begin{aligned}\text{eval}(x) &= x \\ \text{eval}(\lambda x.M) &= \lambda x.M \\ \text{eval}(M N) &= \text{eval}(P[x \backslash N]) \\ &\text{where } \text{eval}(M) = \lambda x.P\end{aligned}$$

- ▶ this corresponds to the big-step call-by-name operational semantics of the λ -calculus
- ▶ it has type $\text{LC} \rightarrow \text{LC}$ (a partial function)
- ▶ it relies on substitution

Closure Conversion

We will now get rid of the expensive substitution function, and turn it into an explicit record (an **environment**) storing the values of the free variables in the program

Wrong attempt:

$$\begin{aligned}\text{bad-cc}(x, e) &= \text{bad-cc}(M, e) \\ &\quad \text{where } e(x) = M \\ \text{bad-cc}(\lambda x.M, e) &= \langle \lambda x.M, e \rangle \\ \text{bad-cc}(M \ N, e) &= \text{bad-cc}(P, e'[x \mapsto N]) \\ &\quad \text{where } \text{bad-cc}(M, e) = \langle \lambda x.P, e' \rangle\end{aligned}$$

where

- ▶ environments have type $\text{Env} = \text{Var} \rightarrow \text{LC}$, where Var is the type of variables
- ▶ this interpreter has type $(\text{LC} \times \text{Env}) \rightarrow (\text{LC} \times \text{Env})$

Why doesn't this work?

Closure Conversion

Consider the following expression:

$$(\lambda y. (\lambda x. (\lambda y. x) \underline{1}) y) \underline{0}$$

where $\underline{0}$ and $\underline{1}$ are the Church numeral representations of 0 and 1.

We are expecting the result to be $\underline{0}$

$$\begin{aligned} & \text{bad-cc}((\lambda y. (\lambda x. (\lambda y. x) \underline{1}) y) \underline{0}, \emptyset) \\ &= \text{bad-cc}((\lambda x. (\lambda y. x) \underline{1}) y, [y \mapsto \underline{0}]) \\ &= \text{bad-cc}((\lambda y. x) \underline{1}, [y \mapsto \underline{0}, x \mapsto y]) \\ &= \text{bad-cc}(x, [x \mapsto y, y \mapsto \underline{1}]) \\ &= \text{bad-cc}(y, [x \mapsto y, y \mapsto \underline{1}]) \\ &= \text{bad-cc}(\underline{1}, [x \mapsto y, y \mapsto \underline{1}]) \end{aligned}$$

Incorrect result. This evaluation implements a **dynamic scoping** strategy rather than **static**.

Closure Conversion

When storing x , we should have remembered the context in which it was evaluated, i.e., $[y \mapsto \underline{0}]$, otherwise when fetching x once y 's binding has been modified, x 's binding does not refer to the correct y (dynamic scoping vs. static scoping).

A **closure** is considered here to be the pair of an expression and an environment of type $\text{Env} = \text{Var} \rightarrow (\text{LC} \times \text{Env})$

Correct attempt:

$$\begin{aligned}\text{cc}(x, e) &= \text{cc}(M, e') \\ &\quad \text{where } e(x) = \langle M, e' \rangle \\ \text{cc}(\lambda x.M, e) &= \langle \lambda x.M, e \rangle \\ \text{cc}(M \ N, e) &= \text{cc}(P, e'[x \mapsto \langle N, e \rangle]) \\ &\quad \text{where } \text{cc}(M, e) = \langle \lambda x.P, e' \rangle\end{aligned}$$

where

- ▶ in the application case, we store $\langle N, e \rangle$ instead of N
- ▶ this interpreter has type $(\text{LC} \times \text{Env}) \rightarrow (\text{LC} \times \text{Env})$

CPS Transformation

The application case makes **nested recursive calls** to the evaluator

A compiler might have to generate a new **stack frame** for those recursive calls in addition to the one it is already using.

A solution: use **tail-recursion**, i.e., recursive calls are only made as the final action of a procedure.

Non tail-recursive:

```
1 f :: Int -> Int -> Int
2 f x y = if x <= 0 then y
3         else 1 + f (x - 1) y
```

Tail-recursive:

```
1 g :: Int -> Int -> Int
2 g x y = if x <= 0 then y
3         else g (x - 1) (y + 1)
```

A stack frame can then be replaced by the one for the tail-recursive call.

Continuations provide a way to transform a recursive function into a tail-recursive function.

CPS Transformation

We now transform the previous evaluator into one that uses a **continuation passing style** (CPS).

This evaluator takes 3 arguments:

- ▶ an expression to evaluate
- ▶ an environment of type $\text{Env} = \text{Var} \rightarrow (\text{LC} \times \text{Env})$
- ▶ a **continuation** of type $\text{Cont} = (\text{Val} \times \text{Env}) \rightarrow (\text{Val} \times \text{Env})$

where Val is the type of values, i.e., λ -abstractions.

A continuation is a function, which specifies what to do next once a term has been evaluated to a value.

$$\begin{aligned}\text{cont}(x, e, k) &= \text{cont}(M, e', k) \\ &\quad \text{where } e(x) = \langle M, e' \rangle \\ \text{cont}(\lambda x.M, e, k) &= k \langle \lambda x.M, e \rangle \\ \text{cont}(M \ N, e, k) &= \text{cont}(M, e, k') \\ &\quad \text{where } k' = \text{fun } \langle \lambda x.P, e' \rangle \Rightarrow \text{cont}(P, e'[x \mapsto \langle N, e \rangle], k)\end{aligned}$$

CPS Transformation

Let us consider the application case:

$$\begin{aligned} \text{cont}(M\ N, e, k) &= \text{cont}(M, e, k') \\ &\text{where } k' = \text{fun } \langle \lambda x. P, e' \rangle \Rightarrow \text{cont}(P, e'[x \mapsto \langle N, e \rangle], k) \end{aligned}$$

This evaluates $M\ N$ w.r.t. a continuation k , which specifies what to do next once the application has been reduced to a value.

It keeps evaluating M w.r.t. a new continuation k' , which specifies that once M has been reduced to a value of the form $\lambda x. P$, then P must be evaluated w.r.t. the extended environment and initial continuation k .

The initial continuation is the identity function: $\text{fun } x \Rightarrow x$

Defunctionalization

The previous evaluator still contains an **anonymous function**, namely the continuation:

$$\text{fun } \langle \lambda x. P, e' \rangle \Rightarrow \text{cont}(P, e'[x \mapsto \langle N, e \rangle], k)$$

We set out to remove higher-order functions, which the CPS transformation phase re-introduced.

Defunctionalization provides a way to transform anonymous functions into first-order data:

- ▶ the information necessary to apply an anonymous function is stored along with a unique tag
- ▶ this tag is used when doing function application to perform the correct action

Defunctionalization

Our simple example has only two anonymous functions:

- ▶ the initial continuation: $\text{fun } x \Rightarrow x$
- ▶ the application continuation:
 $\text{fun } \langle \lambda x.P, e' \rangle \Rightarrow \text{cont}(P, e'[x \mapsto \langle N, e \rangle], k)$

What should the continuation type be?

$$k \in \text{Cont} ::= \text{CONT_I} \mid \text{CONT_}\lambda(\text{LC}, \text{Env}, \text{Cont})$$

where CONT_I is now the initial continuation

The defunctionalized evaluator is defined as follows:

$$\begin{aligned} \text{defunc}(x, e, k) &= \text{defunc}(M, e', k) \\ &\quad \text{where } e(x) = \langle M, e' \rangle \\ \text{defunc}(\lambda x.M, e, k) &= \text{apply_cont}(\lambda x.M, e, k) \\ \text{defunc}(M \ N, e, k) &= \text{defunc}(M, e, \text{CONT_}\lambda(N, e, k)) \end{aligned}$$

where an continuation is applied as follows

$$\begin{aligned} \text{apply_cont}(M, e, \text{CONT_I}) &= \langle M, e \rangle \\ \text{apply_cont}(\lambda x.P, e', \text{CONT_}\lambda(N, e, k)) &= \text{defunc}(P, e'[x \mapsto \langle N, e \rangle], k) \end{aligned}$$

Abstract State Machine

We can now straightforwardly turn this evaluator into an abstract state machine (a variant of Krivine's machine), where:

- ▶ the environment is the heap
- ▶ the continuation is the stack

Note that evaluator can be in two different states:

- ▶ calling `defunc`
- ▶ or calling `apply_cont`

We therefore combine the two tail-recursive functions `defunc` and `apply_cont` into a single one, that uses a switch to distinguish between `defunc` and `apply_cont`.

We introduce accordingly the following states:

$$s \in \text{State} ::= \text{EVAL} \mid \text{CONT}$$

Abstract State Machine

The previous evaluator can be turned into the following one:

$$\begin{aligned}\text{switch}(\text{EVAL}, x, e, k) &= \text{switch}(\text{EVAL}, M, e', k) \\ &\quad \text{where } e(x) = \langle M, e' \rangle \\ \text{switch}(\text{EVAL}, \lambda x.M, e, k) &= \text{switch}(\text{CONT}, \lambda x.M, e, k) \\ \text{switch}(\text{EVAL}, M \ N, e, k) &= \text{switch}(\text{EVAL}, M, e, \text{CONT_}\lambda(N, e, k)) \\ \text{switch}(\text{CONT}, M, e, \text{CONT_I}) &= \langle M, e \rangle \\ \text{switch}(\text{CONT}, \lambda x.P, e', \text{CONT_}\lambda(N, e, k)) &= \text{switch}(\text{EVAL}, P, e'[x \mapsto \langle N, e \rangle], k)\end{aligned}$$

which is of type $(\text{State} \times \text{LC} \times \text{Env} \times \text{Cont}) \rightarrow (\text{LC} \times \text{Env})$

As the defunctionalized evaluator had 5 rules, this one also has 5 rules:

- ▶ 3 EVAL rules
- ▶ 2 CONT rules

We will now:

- ▶ get rid of State by inlining the CONT cases
- ▶ turn continuation into lists of instructions
 - ▶ CONT_I is turned into the empty list $[]$ (the new initial continuation)
 - ▶ $\text{CONT_}\lambda(M, e, k)$ is turned into the list $\langle M, e \rangle :: K$, where K is k 's conversion into a list

Abstract State Machine

A continuation is now of type $\text{Cont} = \text{list}(\text{LC} \times \text{Env})$.

We inline the CONT cases within the λ case.

The previous state-based evaluator is now turned into the following abstract machine:

$$\begin{aligned}\text{asm}(x, e, k) &= \text{asm}(M, e', k) \\ &\quad \text{where } e(x) = \langle M, e' \rangle \\ \text{asm}(\lambda x.M, e, []) &= \langle \lambda x.M, e \rangle \\ \text{asm}(\lambda x.M, e, \langle N, e' \rangle :: k) &= \text{asm}(M, e[x \mapsto \langle N, e' \rangle], k) \\ \text{asm}(M \ N, e, k) &= \text{asm}(M, e, \langle N, e \rangle :: k)\end{aligned}$$

which is of type $(\text{LC} \times \text{Env} \times \text{Cont}) \rightarrow (\text{LC} \times \text{Env})$

This **abstract state machine** (ASM)

- ▶ **pushes** a closure to the stack every time it sees an **application**
- ▶ **pops** a closure from the stack when it sees a **λ -abstraction**, to perform a β -reduction, storing the argument in the heap

Conclusion

What did we cover today?

- ▶ Closure conversion
- ▶ CPS transformation
- ▶ Defunctionalization
- ▶ Krivine's abstract machine